PES UNIVERSITY

Department of Computer Science and Engineering

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Project: Web Based ML Model Evaluator
Document Version: 1.0 Approved

Author	Date	Reason for Change	Version
Team 13	2026-08-31	Initial baseline SRS prepared for the Software Engineering project.	1.0

Table of Contents

- 1. Introduction
- 2. Overall Description
- 3. External Interface Requirements
- 4. Analysis Models
- 5. System Features
- 6. Other Nonfunctional Requirements
- 7. Other Requirements and Initial Validation Planning
- Appendix A: Glossary
- Appendix B: Field Layouts and Report Requirements
- Appendix C: Requirement Traceability Matrix

1. Introduction

1.1 Purpose

This SRS describes the requirements for the Web Based ML Model Evaluator, Version 1.0. It defines what the system is expected to provide to its users, the interfaces it will use, functional requirements, non-functional requirements, security requirements and the information needed for later design and testing.

1.2 Intended Audience and Reading Suggestions

This document is intended for the project team, project evaluators, testers and future maintainers. Readers who want an overview can start with Sections 1 and 2. The functional behaviour is described in Section 5, while non-functional and security requirements are in Section 6. The appendices contain terminology, field information and the initial traceability matrix.

1.3 Product Scope

The system is a web application for evaluating already-trained machine learning models using tabular test datasets. A user uploads a supported model and CSV dataset, the system performs basic validation, executes the model in an isolated environment, calculates evaluation metrics and presents the results through a dashboard. Completed evaluations can be compared and exported as PDF or JSON reports.

Version 1 focuses on classical ML and ONNX models with tabular CSV data. Model training, streaming inference and unstructured media processing are outside the scope of this version.

1.4 References

- PES University, Department of Computer Science and Engineering, Software Engineering – Level 3 Project Guidelines, 2026.
- PES University, Software Requirements Specification template supplied for the project.
- IEEE 830-1998, Software Requirements Specifications.
- Official documentation for scikit-learn and ONNX Runtime.

2. Overall Description

2.1 Product Perspective

The proposed system is an independent client-server web application. The browser-based frontend communicates with a Django REST Framework backend. PostgreSQL is used for application data, while Celery and Redis are used to handle evaluation tasks. Docker is used to isolate model execution.

2.2 Product Functions

- Register and authenticate users.
- Upload supported ML model files and CSV datasets.
- Validate file type, file size and dataset/model feature compatibility.
- Run model evaluation in an isolated execution environment.
- Detect classification or regression tasks and calculate the corresponding metrics.
- Display evaluation results and visual plots.
- Compare results from multiple models evaluated on the same dataset.
- Export evaluation summaries in PDF and JSON formats.
- Maintain a user's evaluation history in a workspace.

2.3 User Classes and Characteristics

User Class	Characteristics / Main Use
Student / Researcher	Non-admin user. Uploads models and datasets, runs evaluations, views results, compares models and exports reports.

User Class	Characteristics / Main Use
Administrator	Higher-privilege user responsible for managing user accounts, system quotas and relevant runtime settings.

2.4 Operating Environment

Client: a modern web browser. Server environment: Linux/Ubuntu 22.04 LTS, Python 3.11, Django, PostgreSQL 15 and Docker. The backend also uses Django REST Framework, Celery and Redis.

2.5 Design and Implementation Constraints

- Model upload size is limited to 200 MB.
- CSV dataset upload size is limited to 100 MB.
- Model execution must take place in a network-disabled Docker container.
- The project uses the proposed React, Django and PostgreSQL based architecture.

2.6 Assumptions and Dependencies

- The dataset contains features compatible with the uploaded model.
- The test dataset contains an appropriate target value for supervised evaluation.
- Required scikit-learn and ONNX Runtime support remains available for the model formats used.
- The server has Docker and the required backend services available.

3. External Interface Requirements

3.1 User Interfaces

The frontend will be implemented as a React single-page application. The main user-facing areas are authentication, file upload, evaluation results/dashboard, model comparison and workspace history. Upload screens should show validation errors clearly, while the dashboard should present metric values and relevant plots without requiring the user to inspect raw program output.

3.2 Software Interfaces

- Django REST Framework APIs exchange structured JSON data with the React frontend.
- PostgreSQL stores users, uploaded model information and evaluation results.
- Celery and Redis handle asynchronous evaluation jobs.
- Docker provides the isolated execution environment for model inference.
- S3 or local file storage may be used for uploaded files as selected during implementation.

3.3 Communications Interfaces

Client-server communication will use HTTPS. File uploads will use multipart/form-data. REST API responses will use JSON. TLS will be used to protect data in transit.

3.4 Hardware Interfaces

No dedicated hardware interface is required for Version 1. The system is intended to run on standard computing infrastructure capable of running the web application and Docker runtime.

4. Analysis Models

The following use-case summary defines the main interactions identified for Version 1. Detailed UML diagrams can be kept consistent with these use cases during the design stage.

UC ID	Use Case	Actor	Precondition	Outcome
UC-01	Register / Login	User	Application is accessible	Authenticated session is created
UC-02	Upload Model and Dataset	User	User is authenticated	Files are validated and saved

UC ID	Use Case	Actor	Precondition	Outcome
UC-03	Execute Evaluation	Worker	Valid files are available	Evaluation metrics are computed
UC-04	View Dashboard	User	Evaluation is completed	Metrics and plots are displayed
UC-05	Compare Models	User	At least two comparable runs exist	Side-by-side comparison is shown
UC-06	Export Report	User	Evaluation is completed	PDF / JSON report is generated

5. System Features

5.1 User Authentication and Workspace Management

Owner: Atharva Anand
Priority: High

Manages registration, authentication, access control and user-specific evaluation history.

Stimulus / Response

User submits credentials → system validates credentials → authenticated session/token is returned → protected resources become available.

Functional Requirements

FR-001: The system shall allow a user to register using an email address and password.

FR-002: The system shall authenticate protected API requests using JWT tokens and reject unauthenticated requests with HTTP 401.

FR-003: The system shall display evaluation history belonging to the authenticated user.

5.2 Dataset Ingestion and Pre-Validation

Owner: Atharva Rakesh Bidre
Priority: High

Handles model and dataset uploads and checks basic compatibility before execution.

Stimulus / Response

User selects files → system checks format and size → system checks dataset/model compatibility → validation status is displayed.

Functional Requirements

FR-004: The system shall accept .pkl, .joblib and .onnx model files up to 200 MB.

FR-005: The system shall accept CSV test datasets up to 100 MB.

FR-006: The system shall check compatibility between dataset feature columns and the uploaded model before execution.

5.3 Sandboxed Model Execution and Metrics

Owner: Ayush M Shetty
Priority: High

Runs model inference in an isolated environment and produces the required evaluation metrics.

Stimulus / Response

Evaluation job is created → worker starts isolated container → model inference is performed → metrics are calculated and stored.

Functional Requirements

FR-007: The system shall execute uploaded model inference inside a network-disabled Docker container.

FR-008: The system shall classify an evaluation as Classification or Regression using the available target information.

FR-009: For classification, the system shall calculate Accuracy, Precision, Recall, F1 and ROC-AUC where applicable; for regression it shall calculate MAE, MSE, RMSE and R^2.

5.4 Benchmarking, Visual Analytics and Report Export

Owner: Bhavesh Velluru
Priority: High

Presents evaluation results, compares completed runs and produces downloadable reports.

Stimulus / Response

User selects a completed run → dashboard loads metrics/plots → user selects comparison or export → requested result/report is generated.

Functional Requirements

FR-010: The system shall display evaluation plots including confusion matrices and ROC curves for applicable classification results, and residual plots for regression results.

FR-011: The system shall display side-by-side metric comparisons for multiple models evaluated on the same dataset.

FR-012: The system shall allow a completed evaluation summary to be exported in PDF and JSON formats.

6. Other Nonfunctional Requirements

6.1 Performance Requirements

NFR-001: Under normal test conditions, evaluation of a dataset containing up to 50,000 rows shall complete within 30 seconds, excluding file upload time.

6.2 Safety Requirements

The evaluation worker shall use a memory limit of 512 MB for the execution container so that a model or dataset using excessive memory is less likely to affect the host service.

6.3 Security Requirements

- **SEC-001:** User passwords shall be stored using a secure password-hashing mechanism provided by the backend framework, using PBKDF2 with SHA-256 as specified for the project.
- **SEC-002:** Uploaded model execution shall occur in a non-root Docker container with network access disabled.
- **SEC-003:** Role-based access control shall be enforced for protected resources.
- **SEC-004:** API input handling shall include validation/sanitization to reduce common SQL injection and XSS risks.

6.4 Software Quality Attributes

- **NFR-002 Usability:** A user should be able to reach the upload-to-results flow in no more than three main interface actions after authentication, excluding file selection.
- **NFR-003 Reliability:** Invalid uploads and failed evaluations shall produce an error state without bringing down the main web service.
- **NFR-004 Availability:** The deployed application should target at least 95% availability during scheduled evaluation/demo periods.
- **NFR-005 Maintainability:** The backend shall be divided into modular Django applications/services, and implemented modules should target at least 80% automated test coverage where feasible.

6.5 Business / Domain Rules

- **BR-001:** A model cannot be evaluated unless a compatible CSV dataset is linked to the evaluation.
- **BR-002:** Uploaded model files that contain detected direct system-execution calls such as os.system shall be rejected during the pre-validation/security checks.
- **BR-003:** Metrics belonging to a completed evaluation run shall not be changed by a normal user.

7. Other Requirements and Initial Validation Planning

7.1 Other Requirements

The system shall retain enough information about each evaluation to identify the model, dataset, task type, calculated metrics and execution time. Detailed database fields are listed in Appendix B.

7.2 Initial Validation Planning

- **Functional validation:** Execute each functional requirement using valid and invalid inputs and compare the actual result with the expected result.
- **Security validation:** Test authorization boundaries and submit controlled malicious or invalid model files to verify that execution remains isolated.
- **Performance validation:** Use synthetic datasets and measure evaluation time against NFR-001.
- **Reliability validation:** Test invalid files, failed evaluation jobs, timeouts and repeated requests to verify that the main service remains available.
- **Reference-output validation:** Compare calculated metrics against known scikit-learn outputs for selected datasets.

Appendix A: Glossary

Term	Meaning
JWT	JSON Web Token used for authorization of protected API requests.
Pickle (.pkl)	Python object serialization format that can be used to store trained model objects.
ONNX	Open Neural Network Exchange format for representing machine learning models.
Sandbox	An isolated execution environment used to reduce the impact of untrusted model execution.
RTM	Requirement Traceability Matrix, linking requirements to design, implementation and tests.

Appendix B: Field Layouts and Report Requirements

B.1 User Table (users_user)

Fields: id (UUID, primary key), email (String 120, unique), password_hash (String 256), full_name (String 100), created_at (DateTime).

B.2 Model Table (models_mlmodel)

Fields: id (UUID, primary key), user_id (UUID, foreign key), model_name (String 100), file_format (Enum), file_path (String 255), uploaded_at (DateTime).

B.3 Evaluation Run Table (evaluations_run)

Fields: id (UUID, primary key), model_id (UUID, foreign key), dataset_name (String 100), task_type (Enum), metrics_json (JSONB), execution_time (Decimal).

B.4 Report Requirements

A generated PDF report should contain the report ID/date, model name, dataset name, summary metrics, applicable evaluation charts and the model comparison matrix when a comparison is requested.

Table	Field	Data Type	Mandatory	Description
users_user	id	UUID	Yes	Unique user identifier / primary key
users_user	email	String(120)	Yes	Login email address
models_mlmodel	id	UUID	Yes	Unique model file identifier
models_mlmodel	file_format	Enum	Yes	.pkl, .joblib or .onnx
evaluations_run	id	UUID	Yes	Unique evaluation run identifier
evaluations_run	metrics_json	JSONB	Yes	Calculated evaluation metrics

Appendix C: Requirement Traceability Matrix (Initial Baseline)

This initial RTM connects the current requirements to the planned architecture/modules and test identifiers. Code file references are based on the current proposed structure and should be updated if implementation names change.

Req ID	Description	Arch Ref	Module / Class	Code File Ref	Test ID	Sys Test ID
FR-001	User Registration	Auth Layer	RegisterView	apps/users/views.py	UT-AUTH-01	ST-AUTH-01
FR-002	JWT Authentication	Auth Layer	JWTAuthMiddleware	apps/users/auth.py	UT-AUTH-02	ST-AUTH-02
FR-003	Evaluation History	Workspace	HistoryView	apps/evaluations/views.py	UT-HIST-01	ST-HIST-01
FR-004	Model Upload	Ingestion	ModelUploadView	apps/models/views.py	UT-FILE-01	ST-FILE-01
FR-005	CSV Upload	Ingestion	DatasetUploadView	apps/models/views.py	UT-FILE-02	ST-FILE-02
FR-006	Schema Validation	Ingestion	SchemaValidator	apps/models/validators.py	UT-FILE-03	ST-FILE-03
FR-007	Sandboxed Execution	Worker	DockerSandboxRunner	services/sandbox.py	UT-EXEC-01	ST-SEC-01
FR-008	Task Type Detection	Worker	TaskDetector	services/evaluator.py	UT-EXEC-02	ST-EXEC-01
FR-009	Metric Calculation	Worker	MetricsCalculator	services/metrics.py	UT-EXEC-03	ST-EXEC-02
FR-010	Visual Dashboard	Analytics	ChartRenderer	apps/evaluations/dashboard.py	UT-VIZ-01	ST-VIZ-01
FR-011	Model Comparison	Analytics	ComparisonService	apps/evaluations/views.py	UT-COMP-01	ST-COMP-01
FR-012	PDF / JSON Export	Analytics	PDFReportExporter	services/pdf_exporter.py	UT-RPT-01	ST-RPT-01
SEC-001	Password Hashing	Security	Auth configuration	config/auth.py	UT-SEC-01	ST-SEC-03
SEC-002	Sandbox Isolation	Security	DockerSecurityPolicy	docker/Dockerfile.worker	UT-SEC-02	ST-SEC-02
SEC-003	RBAC	Security	AuthorizationMiddleware	apps/users/permissions.py	UT-SEC-03	ST-SEC-04
SEC-004	Input Validation	API Layer	InputValidator	apps/core/validators.py	UT-SEC-04	ST-SEC-05
NFR-001	Evaluation < 30s	Worker	CeleryAsyncTaskEngine	evaluations/tasks.py	PERF-01	ST-PERF-01
NFR-002	Upload-to-results usability	Frontend	EvaluationFlow	src/pages/EvaluationFlow.jsx	UT-UX-01	ST-UX-01
NFR-003	Error handling / reliability	Service Layer	ErrorHandler	apps/core/errors.py	UT-REL-01	ST-REL-01
NFR-004	Availability target	Deployment	HealthCheck	deployment/health.py	AVAIL-01	ST-AVAIL-01
NFR-005	Maintainability / coverage	Backend Modules	Test suites	tests/	COV-01	ST-COV-01